

PRD — Backend de Plataforma de Compras (tipo Amazon)

1. Resumen del producto

Se desarrollará un backend en Ruby para una aplicación de e-commerce tipo marketplace. El sistema será responsable de gestionar usuarios, productos, inventario, carrito de compras, órdenes, pagos y administración básica del catálogo.

El backend expondrá una API REST (o GraphQL opcional) que será consumida por apps web y móviles.

2. Objetivo del producto

Construir un backend escalable, seguro y modular que permita:

- Gestión de catálogo de productos
 - Flujo completo de compra (carrito → checkout → orden)
 - Gestión de usuarios y autenticación
 - Control de inventario
 - Procesamiento de órdenes
 - Integración futura con pagos y envíos
-

3. Alcance

Incluido

- API backend en Ruby (preferiblemente Rails API-only)
 - Base de datos relacional (PostgreSQL)
 - Autenticación JWT o similar
 - CRUD de productos, usuarios y órdenes
 - Sistema de carrito de compras
 - Gestión de stock
 - Roles básicos (admin / cliente)
-

4. Usuarios del sistema

- **Cliente:** compra productos, gestiona carrito y órdenes
 - **Administrador:** gestiona productos, inventario y órdenes
 - **Sistema externo (futuro):** pagos, envíos, analítica
-

5. Historias de usuario principales

Cliente

- Como cliente, quiero registrarme e iniciar sesión para comprar productos
- Quiero buscar productos por categoría o nombre
- Quiero agregar productos a un carrito
- Quiero confirmar una compra y generar una orden
- Quiero ver el estado de mis órdenes

Admin

- Quiero crear, editar y eliminar productos
 - Quiero actualizar el stock disponible
 - Quiero ver todas las órdenes del sistema
-

6. Funcionalidades del sistema

6.1 Usuarios y autenticación

- Registro de usuarios
 - Login/logout
 - JWT o sessions token
 - Roles: `customer`, `admin`
-

6.2 Catálogo de productos

- Crear producto
- Editar producto
- Eliminar producto (soft delete recomendado)
- Campos:
 - nombre
 - descripción
 - precio

- stock
 - categoría
 - imágenes (URLs)
 - SKU
-

6.3 Carrito de compras

- Agregar producto al carrito
 - Eliminar producto
 - Actualizar cantidad
 - Vaciar carrito
 - Persistencia por usuario
-

6.4 Órdenes

- Crear orden desde carrito
 - Estados:
 - pending
 - paid
 - shipped
 - delivered
 - cancelled
 - Historial de órdenes por usuario
-

6.5 Inventario

- Descontar stock al confirmar compra
 - Evitar overselling (control de concurrencia)
 - Validación de stock en carrito y checkout
-

6.6 Administración

- Dashboard de órdenes
 - Gestión de productos
 - Gestión de usuarios (opcional MVP+)
-

7. Modelo de datos (propuesta)

Users

- id
- name
- email
- password_digest
- role

Products

- id
- name
- description
- price
- stock
- category_id
- sku
- deleted_at

Categories

- id
- name

Carts

- id
- user_id

CartItems

- id
- cart_id
- product_id
- quantity

Orders

- id
- user_id
- status
- total_price
- created_at

OrderItems

- id
 - order_id
 - product_id
 - quantity
 - price_snapshot
-

8. API Endpoints (REST sugerido)

Auth

- POST `/auth/register`
- POST `/auth/login`
- GET `/auth/me`

Products

- GET `/products`
- GET `/products/:id`
- POST `/products` (admin)
- PUT `/products/:id` (admin)
- DELETE `/products/:id` (admin)

Cart

- GET `/cart`
- POST `/cart/items`
- PUT `/cart/items/:id`
- DELETE `/cart/items/:id`

Orders

- POST `/orders`
- GET `/orders`
- GET `/orders/:id`

Admin

- GET `/admin/orders`
 - GET `/admin/products`
-

9. Requisitos no funcionales

Escalabilidad

- Arquitectura modular (services + controllers)
- Preparado para microservicios futuros

Seguridad

- JWT authentication
- Password hashing (bcrypt)
- Protección contra SQL injection (ORM Rails)
- Rate limiting básico

Performance

- Indexación en tablas clave (products, orders)
- Cache (Redis opcional)
- Queries optimizadas

Confiabilidad

- Transacciones para órdenes
 - Validación de stock atómica
-

10. Stack tecnológico

- Ruby on Rails (API-only recomendado)
 - PostgreSQL
 - Redis (opcional: cache / colas)
 - Sidekiq (jobs async: emails, pagos)
 - RSpec (testing)
-

11. Supuestos

- El frontend será independiente
 - Los pagos se integrarán en una fase posterior de momento serán simulados
 - Se asume un solo marketplace (no multi-vendor en MVP)
-

12. Riesgos

- Concurrencia en stock (overselling)
 - Crecimiento de catálogo sin optimización de búsquedas
 - Complejidad futura en órdenes y pagos
 - Performance si no se cachea correctamente
-